



Segmentation and Compositional Analysis of an EDS Elemental Map in MATLAB

COURSE NAME

Computer Applications to Materials Engineering
(MME - 208)

SUBMITTED TO

Sumit Bhowmick

Lecturer

Department of Materials and Metallurgical Engineering
BUET

SUBMITTED BY

Praggo Pabon

Roll: 2311002

Department of Materials and Metallurgical Engineering
BUET

1 Introduction

Calcium (Ca, red), magnesium (Mg, blue), aluminum (Al, the pale "paste" region surrounding the grains), silicon (Si, green), and sulfur (S, yellow) are the five elemental phases shown by color overlays on an energy-dispersive X-ray spectroscopy (EDS) elemental map that is analyzed in this study. The goal of the assignment is to convert this color-coded image into quantitative composition data by cropping it to the region of interest, using a binary mask to isolate each element, measuring the physical area that each mask covers (using the scale bar to convert pixels to mm²), and expressing each element as a percentage of the total classified area. In order to confirm that the masks clearly divide the five elements, a final check plots the average color of each divided region in 3D RGB space.

The implementation consists of a single MATLAB script, supported by five thresholding functions (`redMask.m`, `blueMask.m`, `alMask.m`, `greenMask.m`, and `yellowMask.m`) that were generated interactively in MATLAB's Color Thresholder app and exported as callable functions. All functions used — `imread`, `imcrop`, `imshowpair`, `regionprops` and `scatter3` — belong to the standard Image Processing Toolbox. This report follows the structure of the code itself.

This micrograph is from the published SEM-EDS characterization of ancient Roman mortar from the Privernum city wall (Seymour et al., Science Advances, 2023), where similar Ca/Si/Al/S maps were used to identify relict lime clasts and their reaction rims.

2 Cropping

The pipeline begins by reading the micrograph with `imread` and displaying it with its on-image annotation text intact (Figure 1). MATLAB's "Import" option was used to import the image.

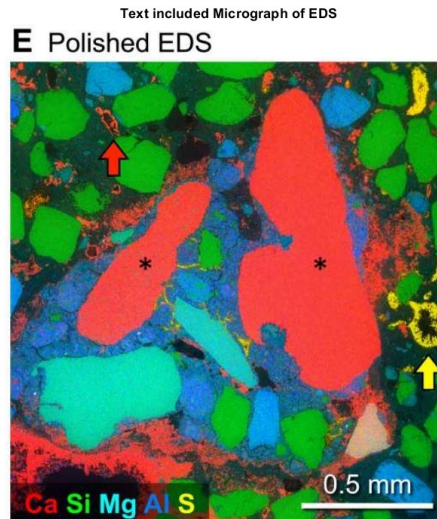


Figure 1: Original EDS elemental map micrograph as imported with `imread`, before cropping.

The crop rectangle `crop` is cached in `coordinates.mat` and reloaded automatically on later runs, with the interactive crop reappearing only if the file is missing (see code comments). `imcrop(EDS, crop)` produces `EDS2` (Figure 2), on which all subsequent masking, measurement, and plotting are performed.

```
1 % first, we are opening the image using imread
2 % mind using your own directory of image while using this command
3 % you can also use 'import' option
4 EDS = imread('C:\Users\Praggo\Downloads\EDS.JPG');
5 % show the image
6 imshow(EDS); figure(1);
7 title('Text included Micrograph of EDS');
8
9 % now we crop it using imcrop. we will make it automatic
10 % because, if we do it manually, then everytime we run the script
11 % the cropping box will come up. so we will do it once.
12
13 % first we will save the coordinates in a file
14 c_file = 'coordinates.mat';
15
16 % now we will do a if-else section which will check if the file
17 % exists, if it does, then it will load the numbers,
18 % if it does not, we will crop it
19
20 if isfile(c_file)
21     load(c_file, 'crop');
22     disp('loaded coordinates for crop...');
23 else
24     disp('could not load coordinates');
25     [~, crop] = imcrop(EDS);
```

```

26     save (c_file,"crop");
27     disp ('coordinates saved...');
28 end
29
30 % now we crop the image from the saved coordinates files
31 % we crop it manually
32 EDS2 = imcrop (EDS, crop); figure(2);
33 imshow (EDS2); title("Cropped EDS");

```

Listing 1: Reading, coordinate caching, and cropping of the EDS micrograph

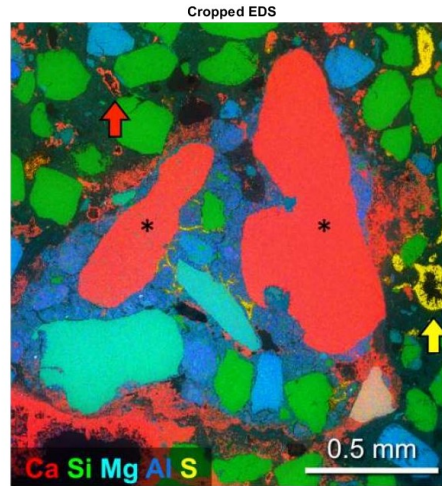


Figure 2: Cropped EDS micrograph (EDS2),

3 Masking: Ca, Mg, Al, Si and S

Each element is separated from the rest of the image by color thresholding. For every mask color, the Color Thresholder app was used to inspect the histograms of the image's channels in a chosen color space and to bracket — by eye — the range of values occupied by that color; then exported a standalone thresholding function for each mask. Every exported function follows the same three-step pattern. The composite image is what makes the montage comparisons of Figures 3–7 possible.

In terms of standard thresholding techniques, all five masks are global, manually supervised box thresholds applied in a transformed color space. The channel windows were tuned interactively against the visual appearance of each mask and verified before export.

Three different color spaces appear across the five masks, each chosen by segmentation quality for that particular color. It was done manually, so I just chose the best possible selection method for each section.

Table 1: Summary of the five element masks

Element	Mask color	Function	Color space
Ca	Red	redMask.m	HSV
Mg	Blue	blueMask.m	L*a*b*
Al	Paste	alMask.m	YCbCr
Si	Green	greenMask.m	L*a*b*
S	Yellow	yellowMask.m	L*a*b*

```

1 % now that we have cropped EDS, we will work on EDS2
2 % we used Color Threshold app and used L*a*b* (as we found it
3 % suitable) to extract yellow part from the cropped EDS
4 % we also exported a function named it yelloMask.m
5 % now we call the function
6 bwEDS = yellowMask(EDS2); figure(3);
7 imshowpair(EDS2, bwEDS, "Montage");
8 title("Comparison between EDS and yellow filtered EDS");
9
10 % we do that with other colors too
11 % for red/pink
12 bwEDS2 = redMask(EDS2); figure(4);
13 imshowpair(EDS2, bwEDS2, "Montage");
14 title("Comparison between EDS and red filtered EDS");
15
16 % for green
17 bwEDS3 = greenMask(EDS2); figure(5);
18 imshowpair(EDS2, bwEDS3, "Montage");
19 title("Comparison between EDS and green filtered EDS");
20
21 % for paste/ aluminum color
22 bwEDS4 = alMask(EDS2); figure(6);
23 imshowpair(EDS2, bwEDS4, "Montage");
24 title("Comparison between EDS and paste filtered EDS");
25
26 % for blue/Magnesium
27 bwEDS5 = blueMask(EDS2); figure(7);
28 imshowpair(EDS2, bwEDS5, "Montage");
29 title("Comparison between EDS and blue filtered EDS");

```

Listing 2: The five exported mask functions on the cropped image

3.1 Calcium (Ca) — Red Mask (HSV)

Table 2: HSV threshold window used by redMask.m (Ca). The hue channel is tested with a logical OR between the two windows.

Channel	Min	Max	Interpretation
H (upper window)	0.808	1.000	Red hues approaching 360° ($\approx 291^\circ$ – 360°)
H (lower window)	0.000	0.071	Red hues just past 0° ($\approx 0^\circ$ – 26°)
S	0.243	0.856	Excludes greys/paste (too low) and saturated overlays (too high)
V	0.204	1.000	Excludes near-black annotation marks

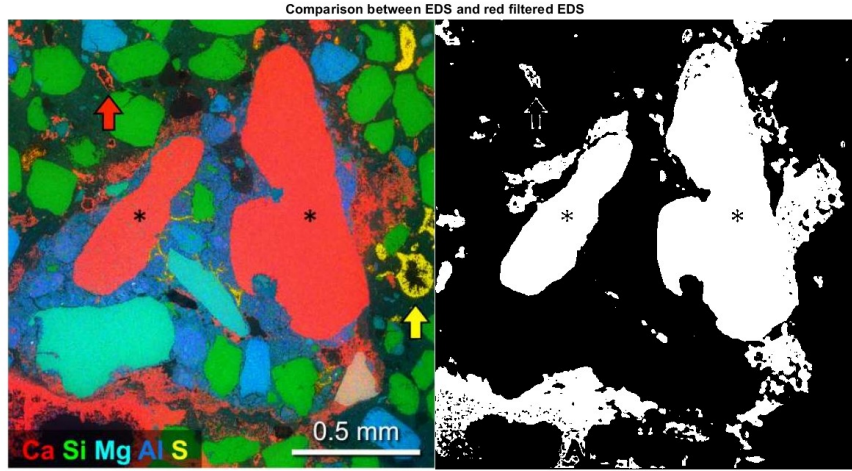


Figure 3: Visual verification of the Ca mask: comparison between the cropped EDS and the red-filtered (Ca) binary mask.

3.2 Magnesium (Mg) — Blue Mask ($L^*a^*b^*$)

Table 3: $L^*a^*b^*$ threshold window used by `blueMask.m` (Mg).

Channel	Min	Max	Interpretation
L^*	67.776	84.480	Light-to-very-light blues only
a^*	-84.682	-21.282	Green side of the green-red axis
b^*	-41.892	16.394	Blue side of the blue-yellow axis (separates Mg from Si)

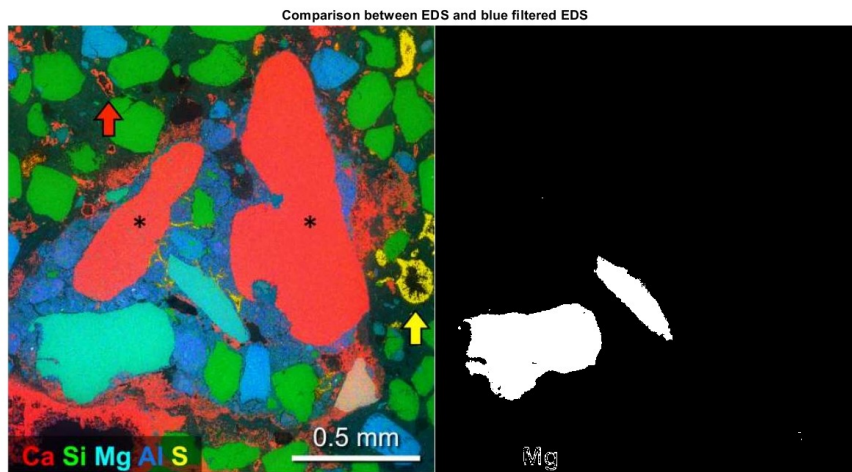


Figure 4: Visual verification of the Mg mask: comparison between the cropped EDS and the blue-filtered (Mg) binary mask.

3.3 Aluminum (Al) — Paste Mask (YC_bC_r)

Table 4: YC_bC_r threshold window used by `alMask.m` (Al).

Channel	Min	Max	Interpretation
Y	57	122	Mid-range luminance: pale paste, not bright white, not dark grains
C_b	131	221	Chroma biased toward blue (above the neutral value of 128)
C_r	0	108	Chroma biased away from red (below the neutral value of 128)

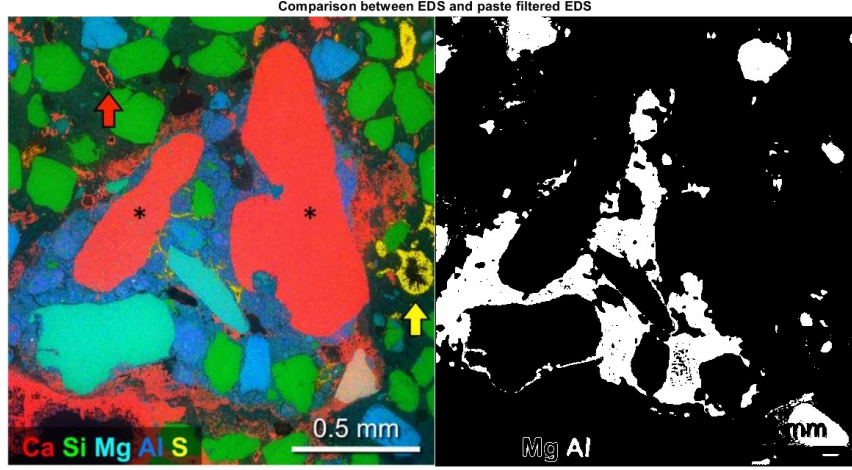


Figure 5: Visual verification of the Al mask: comparison between the cropped EDS and the paste-filtered (Al) binary mask.

3.4 Silicon (Si) — Green Mask ($L^*a^*b^*$)

Table 5: $L^*a^*b^*$ threshold window used by `greenMask.m` (Si).

Channel	Min	Max	Interpretation
L^*	0.040	71.590	Dark-to-mid greens
a^*	-84.682	-21.316	Strongly green (separates Si from Ca and S)
b^*	17.105	65.526	Warm-tinted greens; excludes cool blue-greens

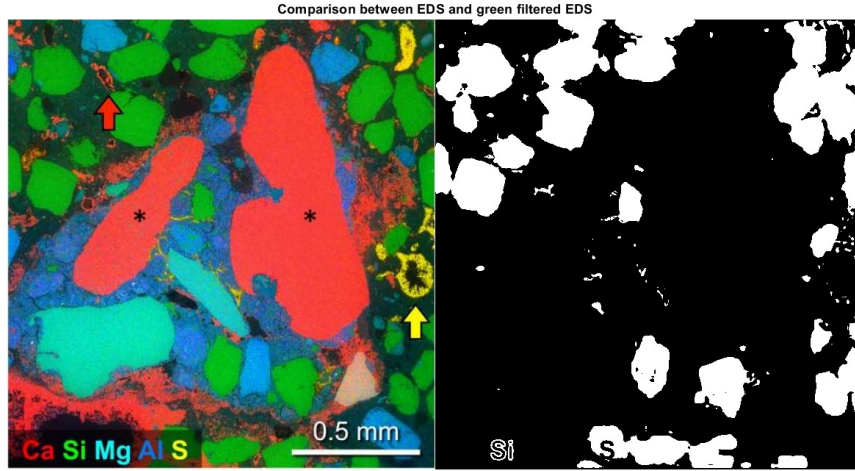


Figure 6: Visual verification of the Si mask: comparison between the cropped EDS and the green-filtered (Si) binary mask.

3.5 Sulfur (S) — Yellow Mask ($L^*a^*b^*$)

Table 6: $L^*a^*b^*$ threshold window used by yellowMask.m (S).

Channel	Min	Max	Interpretation
L^*	7.180	89.615	Broad luminance range: yellows at almost any brightness
a^*	-23.593	26.190	Near-neutral; admits green- and orange-leaning yellows
b^*	37.446	94.288	Strongly yellow (primary discriminator for S)

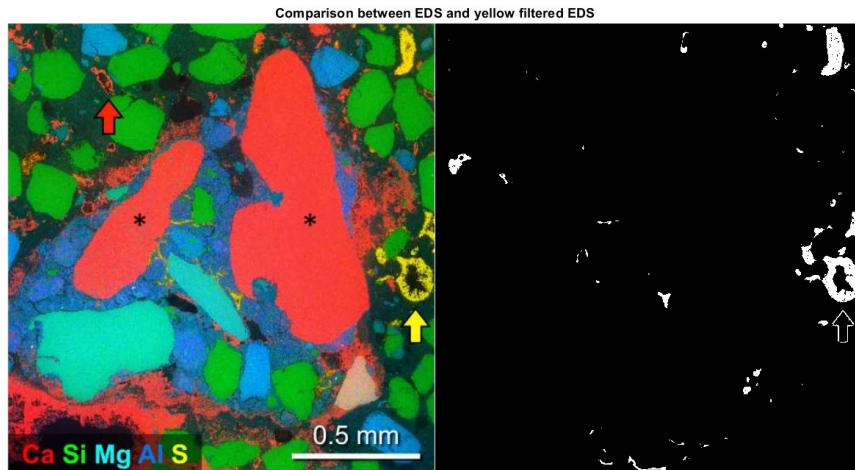


Figure 7: Visual verification of the S mask: comparison between the cropped EDS and the yellow-filtered (S) binary mask.

4 Fraction of Regions Output

The measured areas are in pixels and must be converted to physical units. To this end, the 0.5 mm scale bar on the micrograph was measured once in the Image Tool (`imtool`) ruler at 173.5 px. Since area scales with the square of the linear scale factor, every pixel area is multiplied by the conversion factor

$$c_f = \left(\frac{0.5 \text{ mm}}{173.5 \text{ px}} \right)^2 \approx 8.3 \times 10^{-6} \text{ mm}^2/\text{px}, \quad (1)$$

so that each element's physical area is $A_X^{\text{mm}} = A_X^{\text{px}} \cdot c_f$. The five converted areas are printed to the console with `disp`, in the order S, Ca, Si, Al, and Mg (Listing 3).

Finally, each element is reported as a fraction of the total classified area. The denominator is the *sum of the five element areas*, not the area of the full cropped micrograph: pixels that fall outside all five threshold windows are, by construction, not confidently assigned to any element, and including them would weaken the claim being made (Section 7). Formally, for $X \in \{\text{S, Ca, Si, Al, Mg}\}$,

$$f_X = \frac{A_X}{A_S + A_{\text{Ca}} + A_{\text{Si}} + A_{\text{Al}} + A_{\text{Mg}}}, \quad (2)$$

and the script prints each $f_X \times 100\%$ to the console (Listing 4).

Table 7 is provided for recording the numerical output of a script run.

```

1 % regionprops gives properties of segmented regions
2 prop1 = regionprops("table", bwEDS, "Area");
3 % sum of white pixels in the segmented area
4 area1 = sum(prop1.Area);
5 % convert pixels to mm^2 using the image tool ruler:
6 % 173.5 pixels = 0.5 mm, so the conversion factor is (0.5/173.5)^2
7 cf = (0.5/173.5)^2;
8 % area of S in mm^2
9 disp("Area of S in mm^2");
10 mmarea1 = area1*cf; disp(mmarea1);
11 % repeat the above for bwEDS2..bwEDS5 (Ca, Si, Al, Mg)

```

Listing 3: Example part of code for one element

```

1 % so the total area
2 area = mmarea1 + mmarea2 + mmarea3 + mmarea4 + mmarea5;
3
4 % fraction of S:
5 fS = mmarea1 / area;
6 disp("Fraction percentage of S: "); disp(fS*100);
7 % fraction of Ca:

```

Listing 4: Fraction of one element relative to the total classified area

Table 7: Measured areas and composition fractions

Element	Mask color	Area (mm ²)	Fraction (%)
S	Yellow	0.0513	2.4278
Ca	Red	0.8650	40.8948
Si	Green	0.5874	27.7709
Al	Paste	0.4391	20.7598
Mg	Blue	0.1723	8.1467
Total (classified)	—	2.1151	100.00

The composition is consistent with Seymour et al.’s (2023) description of a relict lime clast: their reaction-rim structure is mirrored by the asterisk-marked calcium-rich core (41% Ca) with sulfur restricted to thin boundary rims (2.4% S). They suggest that these clasts serve as a long-term calcium reservoir, providing a mechanical explanation for the longevity of Roman concrete.

5 Average Region Colors

Before 3D plotting, each connected region in every thresholded mask is characterized by its mean R, G, and B value in the original cropped image EDS2, giving an independent mask check. The five masks and labels are stored in `masks` and `labels`; channels are split once (cast to `double`). A `minArea` = 30 pixel filter removes small regions only in this verification/plotting stage, while area statistics in Section 4 use raw, unfiltered masks (Section 7). For each surviving region, `regionprops` with `PixelIdxList` indexes the R, G, and B arrays; means are divided by 255 to give `regionRGB`, and valid region counts are stored in `regionCounts` and printed.

```

1  % for each connected region found in each thresholded mask, find
2  % the MEAN R,G,B of that region as it appears in the original
3  % (unthresholded) cropped image EDS2.
4  masks = {bwEDS, bwEDS2, bwEDS3, bwEDS4, bwEDS5};
5  labels = {'S', 'Ca', 'Si', 'Al', 'Mg'};
6
7  % discard tiny/noise regions (stray pixels from thresholding, thin
8  % boundary slivers, leftover text/arrow fragments etc.)
9  minArea = 30;
10
11 % now we split the original cropped image into channels once
12 R = double(EDS2(:,:,1));
13 G = double(EDS2(:,:,2));
14 B = double(EDS2(:,:,3));
15
16 figure(8); hold on;
17 regionCounts = zeros(1, numel(masks));
18
19 for k = 1:numel(masks)
20     stats = regionprops(masks{k}, 'PixelIdxList', 'Area');
21     % drops noise regions
22     stats = stats([stats.Area] >= minArea);
23     regionCounts(k) = numel(stats);
24     nReg = numel(stats);
25     regionRGB = zeros(nReg, 3);

```

```

26
27     for r = 1:nReg
28         % linear pixel indices of this region
29         idx = stats(r).PixelIdxList;
30         regionRGB(r,:) = [mean(R(idx)), mean(G(idx)), mean(B(idx))
31     ] / 255;
32     end

```

Listing 5: Per-region average color computation inside the per-element loop

6 3D Plotting

Scatter plot is accumulated element by element. The existing loop already visits one mask/label at a time and computes `regionRGB` for that element, so `scatter3` is called inside the loop to add those regions to the same axes. A `hold on` is therefore implied before the loop; without it, each new call would replace the previous element's points. Passing `regionRGB` as the color argument means each point is painted with its own measured mean RGB, so no artificial color mapping is needed and clustering can be checked visually.

```

1     % each point is colored by its OWN measured average color, so
2     % the
3     % plot visually shows clustering without needing a separate
4     % legend
5     scatter3(regionRGB(:,1), regionRGB(:,2), regionRGB(:,3), 50,
6     ...
7     regionRGB, 'filled', 'MarkerEdgeColor', 'k');
8 end
9
10 xlabel('Mean R (normalized)');
11 ylabel('Mean G (normalized)');
12 zlabel('Mean B (normalized)');
13 title('3D scatter of per-region average colors (clustering by
14 element)');
15 legend(labels, 'Location', 'bestoutside'); grid on; view(135, 25);
16 hold off;
17
18 disp('Number of valid regions counted per element:');
19 for k = 1:numel(masks)
20     fprintf('%s: %d regions\n', labels{k}, regionCounts(k));
21 end

```

Listing 6: 3D scatter of per-region average colors and per-element region counts

Table 8: Number of valid (connected) regions per element

Element	Valid regions (≥ 30 px)
S	24
Ca	48
Si	41
Al	52
Mg	6

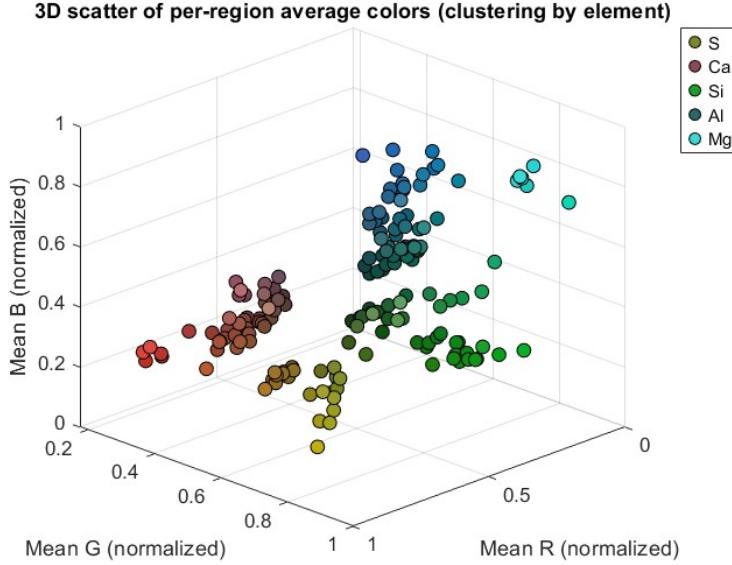


Figure 8: 3D scatter of per-region average colors in normalized RGB space. Points are colored by their own measured average color; same-element regions are expected to form tight, well-separated clusters.

7 Sources of Error and Design Tradeoffs

The following limitations are acknowledged explicitly rather than silently ignored. Each item states what was or was not done, why that decision was made and what bias it may introduce into the measured areas and fractions.

Arrow and asterisk regions were not masked out. I chose not to add a manual exclusion step because it would require redrawing for every new image run through this pipeline, which works against the goal of an automated, reusable script. The arrows and asterisks occupy a visually small footprint relative to the grains they are pointing at. So I judged the resulting bias to be small because some of the colored sections were getting black through the manual thresholding process - both will cancel each other more or less. Though I have not quantified it precisely and treat it as an acknowledged limitation rather than a corrected one.

No minimum region size filter was applied. `regionprops` was run directly on the raw thresholded masks without removing small spurious regions from JPEG compression artifacts or anti-aliased grain boundaries. I left this out because choosing a cutoff is itself a judgment call with no principled answer from the image alone, and I preferred not to introduce an arbitrary threshold that I could not justify quantitatively. This means total area sums include some noise contribution, though for large-area statistics like total Ca or Si area this contribution is likely minor relative to the well resolved grains.

Fraction was computed relative to total classified area, not total image area. I divided each element's area by the sum of the five classified areas rather than by the full cropped micrograph area. This was a deliberate choice. Pixels outside

all five thresholds not confidently assigned to any element, so including them in the denominator would mean reporting a fraction “of the image” using data I don’t actually trust as element-specific. The tradeoff is that this reports composition among classified material rather than across the whole field of view, which is a narrower but more defensible claim given the thresholding quality.

Yellow and green masks have a small overlap in threshold space. The $L^*a^*b^*$ ranges chosen for S (yellow) and Si (green) overlap slightly at their boundary, so a handful of pixels could in principle satisfy both. I did not add mutual exclusion logic between masks because the overlap region is narrow and I expect very few pixels to actually fall in it. But I have not verified this by directly counting overlapping pixels, so I am flagging it as an unverified assumption.

The Al mask was thresholded in a different color space (YC_bC_r) than the other masks. The Al mask was thresholded in YC_bC_r , whereas the S, Si, and Mg masks used $L^*a^*b^*$ and the Ca mask used HSV. This was a practical choice made in the Color Thresholder app when $L^*a^*b^*$ did not cleanly separate the Al/paste color from the background, YC_bC_r gave a visually cleaner mask for that specific color. I accepted the inconsistency in color spaces across masks in exchange for better segmentation quality on that particular element.

Pixel to mm calibration relied on a single manual scale-bar measurement. I measured the 0.5 mm scale bar once using the Image Tool cursor (173.5 px) rather than averaging multiple measurements. I judged the subpixel placement error here to be small relative to the resolution of the regions being measured, so I used the single measurement rather than adding repeated-measurement averaging.